

COMP3308/3608 Week 2: Problem Solving and Search

Kenny Yu

2026 Semester 1

Contents

1	Search Problem Formulation	3
2	Abstraction: Choosing States and Actions	3
3	Canonical Examples of Formulation	4
4	Searching for Solutions: Trees, Nodes, and the Fringe	5
5	Generic Tree-Search Algorithm	5
5.1	High-Level Description	5
5.2	More Detailed Pseudocode (Tree Search)	5
6	Repeated States and Loops (Why Tree Search Can Fail)	6
6.1	Graph Search Idea (Conceptual Extension)	6
7	Evaluating Search Strategies	7
8	Uninformed (Blind) Search	7
8.1	Breadth-First Search (BFS)	7
8.2	Uniform-Cost Search (UCS)	8
8.3	Depth-First Search (DFS)	9

8.4	Depth-Limited Search (DLS)	10
8.5	Iterative Deepening Search (IDS)	10
9	Informed (Heuristic) Search	11
9.1	Uninformed vs Informed	11
9.2	Best-First Search (General Pattern)	11
9.3	Greedy Best-First Search (Greedy Search)	12
10	Connecting the Cost Functions: $g(n)$ vs $h(n)$	12
11	Implementation Notes	13
11.1	Data Structure required	13
11.2	Repeated-State Checking	13
12	Summary	13

Many AI tasks can be expressed as:

- We are in some **initial state**.
- We want to reach a **goal state** (or any state satisfying a goal test).
- We can apply **operators/actions** that transform one state into another.
- We want a **solution**: a sequence of actions (equivalently, a path in a state-space graph).

E.g. Driving, Chess, Reasoning.

1 Search Problem Formulation

Definition 1.0.1 (Search Problem). *A search problem is defined by 4 items:*

1. **Initial state** s_0
2. **Goal state(s)** G (explicit list of implicit goal test)
3. **Operators/actions** $A(s)$: for each state s , the set of legal actions and their outcomes
4. **Path cost function** $g(\cdot)$: assigns a numeric cost to a path (e.g. distance, time, money, etc.)

A **solution** is a path from s_0 to some $s \in G$. An **optimal solution** is one with **minimum path cost**.

Definition 1.0.2 (State Space). *The **state space** is the set of all states reachable from initial state s_0 by applying operators. It is often convenient to view the problem as a graph*

$$G = (V, E),$$

where the vertices V is the set of all states, and edges E is the set of all actions (often with step costs).

2 Abstraction: Choosing States and Actions

Real-world problems are too complex to solve directly. We must **abstract**:

- Remove irrelevant details (e.g., weather, scenery for route planning).
- Choose action granularity that is meaningful (e.g., "drive Arad $\rightarrow$ Sibiu" rather than micro-steering actions).

The level of abstraction must be valid and useful—too fine-grained explodes the state space; too coarse may remove necessary distinctions.

3 Canonical Examples of Formulation

Example 3.0.1 (Romania Route-Finding). Goal: be in Bucharest when currently in Arad.

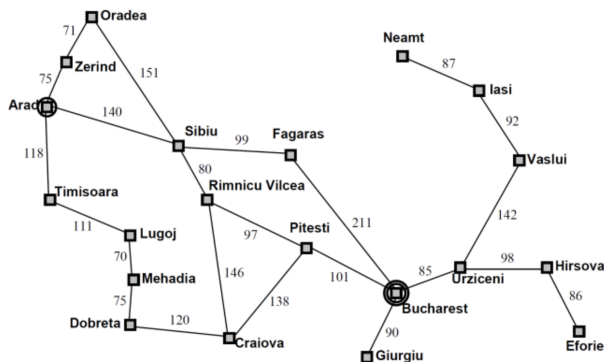


Figure 1: Romania map.

- States: cities
- Actions: drive along roads (e.g., $\text{Arad} \rightarrow \text{Sibiu}$, $\text{Arad} \rightarrow \text{Zerind}$)
- Path cost: road distance/time/etc.

Example 3.0.2 (8-Puzzle). We have following:

- State: a board configuration
- Operators: move blank left/right/up/down
- Path cost: often 1 per move $\rightarrow$ total cost = number of steps

Example 3.0.3 (8-Queens). GOal: place 8 queens so none attack each other. We have two "incremental" formulations:

1. Naive incremental:

- States: any arrangement of $0 \dots 8$ queens anywhere
- Operator: add a queen to any square
- State space huge: $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$. Improvement idea: forbid placing queens in attacked squares.

2. Structured incremental:

- States: $0 \dots 8$ queens, **one per column**, with no attacks
- Operator: place a queen in the **left-most empty column** in a non-attacked row
- State space drastically smaller (2057 states, stated as proven)

Takeaway: Better state/action design can reduce search by astronomical factors.

4 Searching for Solutions: Trees, Nodes, and the Fringe

Search Tree vs State-Space Graph

We often "superimpose" a **search tree** over the underlying state-space graph:

- Tree nodes correspond to **paths** (and thus contain extra metadata).
- Graph states are "pure world configurations".

Nodes vs States

A **state** is a configuration of the world. A **node** is a data structure in the search tree that points to a state, and stores parent pointer, action taken, depth, and path cost g , etc.

This matters because the *same state* can appear in the search tree multiple times via different paths. Nodes let us reconstruct the final solution path by backtracking parent pointers.

Expanded List and Fringe

Expanded: nodes already expanded (we generated their children).

Fringe/Frontier: nodes generated but not yet expanded.

The fringe is kept ordered (often a priority queue) so that the "best" candidate is selected next according to the search strategy.

5 Generic Tree-Search Algorithm

5.1 High-Level Description

Core loop:

- Choose a node from the fringe (based on strategy).
- If it is a goal, return solution.
- Else expand it (generate successors/children) and update fringe.

5.2 More Detailed Pseudocode (Tree Search)

```

1 TREE-SEARCH(problem, strategy) -> solution OR failure
2   fringe <- empty priority queue / deque (depends on strategy)
3   fringe.insert( MAKE-NODE(problem.initial_state) )
4
5   while fringe not empty:
6     node <- fringe.pop_best(strategy)    // "remove front" in slides
7     if problem.goal_test(node.state):
8       return SOLUTION(node)             // backtrack parents
9     successors <- EXPAND(node, problem)
10    fringe.insert_all(successors, strategy_order)
11    return failure
12
13 EXPAND(node, problem) -> set of child nodes
14   successors <- empty set
15   for each action in problem.actions(node.state):
16     s' <- problem.result(node.state, action)
17     child <- new Node(state=s')
18     child.parent <- node
19     child.action <- action
20     child.depth <- node.depth + 1
21     child.g <- node.g + problem.step_cost(node.state, action, s')
22     add child to successors
23   return successors

```

6 Repeated States and Loops (Why Tree Search Can Fail)

In many problems (e.g., Romania), we can return to the same city, creating **cycles**. BFS/DFS on a pure tree can:

- revisit states repeatedly
- potentially loop forever (especially DFS)

6.1 Graph Search Idea (Conceptual Extension)

To handle repeated states, practical search often uses **graph search** (aka repeated-state checking):

- Maintain a "visited/closed" set of states already expanded (and sometimes also in fringe).
- Don't enqueue successors whose state has already been seen.

A typical version:

```

1 GRAPH-SEARCH(problem, strategy) -> solution OR failure
2   fringe <- { initial node }
3   closed <- empty set
4
5   while fringe not empty:
6     node <- pop_best(fringe)

```

```

7         if goal_test(node.state):
8             return SOLUTION(node)
9         if node.state in closed:
10            continue
11        add node.state to closed
12        for child in EXPAND(node):
13            if child.state not in closed:
14                insert child into fringe (strategy)
15    return failure

```

7 Evaluating Search Strategies

A search strategy defines which fringe node is "most promising" and thus expanded next.

Four criteria:

1. **Completeness:** guaranteed to find a solution if one exists?
2. **Optimality:** guaranteed to find a least-cost function?
3. **Time complexity:** number of generated nodes (proxy for runtime)
4. **Space complexity:** maximum number of nodes stored in memory

Complexities often expressed in terms of:

- b : maximum branching factor
- d : depth of an optimal (least-cost) solution (or shallowest optimal under unit costs)
- m : maximum depth of the state space (finite or infinite)

8 Uninformed (Blind) Search

Uninformed strategies do **not** use domain-specific knowledge about distance-to-goal; they only know goal vs non-goal when they tests a node.

8.1 Breadth-First Search (BFS)

BFS expands the shallowest unexpanded node first. This corresponds to exploring the search tree level by level.

Implementation Mechanism. Use a FIFO queue discipline:

- Pop from front of fringe
- Insert generated children at **end** of fringe

```

1 BFS(problem) -> solution OR failure
2   fringe <- FIFO queue
3   fringe.enqueue(initial_node)
4   while fringe not empty:
5       node <- fringe.dequeue()
6       if goal_test(node.state):
7           return SOLUTION(node)
8       for child in EXPAND(node):
9           fringe.enqueue(child)
10  return failure

```

Completeness: Yes, if b is finite. BFS explores all nodes at depth $0, 1, 2, \dots$ so if a solution exists at some finite depth, BFS will eventually reach it.

Optimality: If **all step costs are equal** (e.g., each action costs 1), BFS is optimal: the first goal found is at minimum depth, hence minimum cost. If step costs vary, BFS is **not** optimal because "shallowest" does not imply "least-cost".

Time complexity: Number of generated nodes up to depth d :

$$1 + b + b^2 + \dots + b^d = O(b^d)$$

Space complexity: Also $O(b^d)$, because BFS stores essentially the whole frontier of the deepest level reached (and typically many prior nodes too).

8.2 Uniform-Cost Search (UCS)

BFS ignores step cost; it may return a shallow but expensive solution.

UCS expands the node with the lowest path cost $g(n)$ from the root. So UCS prioritises "cheapest-so-far".

Implementation Mechanism. Use a priority queue ordered by $g(n)$:

- Dequeue node with minimum $g(n)$
- Insert successors with their updated g values, maintaining the priority ordering.

```

1 UCS(problem) -> solution OR failure
2   fringe <- min-priority-queue keyed by g
3   insert initial_node with g=0
4   while fringe not empty:
5       node <- fringe.dequeue()
6       if goal_test(node.state):
7           return SOLUTION(node)
8       for child in EXPAND(node):
9           fringe.enqueue(child)
10  return failure

```


In graph-search form, UCS additionally must handle the case where a cheaper path to an already-seen state is found.

Completeness: Yes, assuming every step cost > 0 . With positive costs, we cannot keep generating infinitely many nodes with total cost bounded by a constant; thus UCS will eventually dequeue all nodes cheaper than the optimal solution cost and then reach the optimum.

Optimality: Yes, because UCS always expands the fringe node with smallest g . When it dequeues a goal node, that goal has the smallest possible cost among all fringe candidates; with positive costs, no undiscovered cheaper goal can exist.

Time/Space complexity: Both time and space complexity of

$$O(b^{1+\lceil C^*/\epsilon \rceil}),$$

where C^* is the cost of the optimal solution, ϵ is the minimum step cost. UCS may explore many "tiny-step" paths (low increments in g) before trying bigger-step paths that might be more direct.

The UCS and BFS return the same solution when:

- step costs constant, or
- $g(n)$ essentially increases with depth in the same way.

If all actions have identical cost, "least g " is the same as "least depth".

8.3 Depth-First Search (DFS)

DFS expands the deepest unexpanded node first. It explores one branch as far as possible before backtracking.

Implementation Mechanism. Use a LIFO stack discipline:

- Pop from front/top of fringe.
- Push children at the **front** of fringe.

```
1 DFS(problem) -> solution OR failure
2   fringe <- stack
3   fringe.push(initial_node)
4   while fringe not empty:
5       node <- fringe.pop()
6       if goal_test(node.state):
7           return SOLUTION(node)
8       for child in EXPAND(node):
9           fringe.push(child)    // so it gets explored next
10  return failure
```

Completeness: Not complete in infinite-depth spaces ($m = \infty$), because it might go down an infinite path and never come back. Complete if the maximum depth m is finite.

Optimality: No. DFS can find a longer (higher-cost) goal before a better one.

Time complexity: Worst case $O(b^m)$, because it may traverse to maximum depth and explore almost everything.

Space complexity: $O(bm)$ (linear in depth), which is dramatically better than BFS's exponential memory. Once a node's descendants are fully explored, the node can be discarded—DFS mostly stores the current path plus some siblings.

Practical warning: DFS can get stuck in cycles without repeated-state checking.

8.4 Depth-Limited Search (DLS)

DLS is DFS with a depth cutoff l . It refuses to expand nodes deeper than l .

```
1 DLS(node, l)
2     if goal_test(node.state):
3         return SOLUTION(node)
4     else if node.depth == l:
5         return cutoff
6     else:
7         for child in EXPAND(node):
8             result <- DLS(child, l)
9             if result is solution:
10                 return result
11         return failure
```

Completeness: Yes **for the explored depth**, since search depth is finite. (But note: if the true solution depth $> l$, DLS won't find it.)

Optimality: Similar to DFS, no.

Time complexity: $O(b^l)$

Space complexity: $O(bl)$

Choosing l : If we know an upper bound on solution depth (e.g., Romania has 20 cities, then any simple path ≤ 19), we can pick l accordingly. But often the "diameter" or max needed depth is unknown in advance.

8.5 Iterative Deepening Search (IDS)

IDS repeatedly runs depth-limited DFS with increasing limits: $l = 0, 1, 2, \dots$ until it finds a goal.

```
1 IDS(problem):
2     for l = 0 to infinity:
```

```

3      result <- DLS(initial_node, 1)
4      if result is solution:
5          return result

```

Ti combines:

- DFS's low memory usage
- BFS's completeness (for finite branching) and BFS's optimality *when step cost is uniform*

Even though nodes near the root are expanded multiple times, the overhead is small because most nodes are at the deepest level d , not near the root.

Expansion Overhead (Quantitative Intuition). Repeated expansions happen mostly at shallow levels, which contain far fewer nodes than deep levels.

Completeness: Yes (as BFS, under finite b).

Optimality: Same as BFS: not in general; yes if step cost = 1.

Time complexity: $O(b^d)$ (like BFS)

Space complexity: $O(bd)$ (like DFS)

9 Informed (Heuristic) Search

9.1 Uninformed vs Informed

Uninformed methods:

- expand systematically (e.g., BFS level-by-level)
- only know "goal or not" when tested
- don't know how far the goal is

Informed methods use **heuristic knowledge**—an estimate of distance-to-goal—to choose which node to expand.

9.2 Best-First Search (General Pattern)

Informed search compares non-goal nodes using an **evaluation function** that estimates how good a node is.

At each step: expand the node that *appears* best according to that evaluation function (since it's only an estimate).

9.3 Greedy Best-First Search (Greedy Search)

Greedy Search uses the heuristic value $h(n)$ as its evaluation:

- $h(n)$ = estimated cost from node n to a goal.
- Expand the node with **minimum** $h(n)$ (appears closest to goal).
- For a goal node, $h(\text{goal}) = 0$.

Example 9.3.1 (Romania). A classic heuristic is straight-line distance to Bucharest:

$$h(n) = \text{SLD}(n, \text{Bucharest})$$

```
1 GREEDY-BEST-FIRST(problem, h) -> solution OR failure:
2   fringe <- min-priority-queue keyed by h(node)
3   fringe.enqueue(initial_node)
4   while fringe not empty:
5     node <- fringe.dequeue
6     if goal_test(node.state):
7       return SOLUTION(node)
8     for child in EXPAND(node):
9       fringe.enqueue(child)
10  return failure
```

Greedy focuses purely on "estimated remaining distance" and ignores cost already spent $g(n)$, so it can rush toward the goal-looking region quickly, but may take expensive steps.

In the Romania example, it finds:

Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

with cost 450, but there exists a shorter 418 route (so greedy is not optimal).

Completeness: Like DFS, yes in finite spaces (finite m), no in infinite spaces; can also get stuck in loops without repeated-state checking.

Optimality: No

Time: $O(b^m)$ worst-case, but good heuristics can dramatically improve practical performance.

Space: $O(b^m)$ (keeps many nodes in memory).

10 Connecting the Cost Functions: $g(n)$ vs $h(n)$

Recall that $g(n)$ denotes cost so far (from start to n), and $h(n)$ is the estimated remaining cost (from n to goal).

Greedy uses $h(n)$. If we (incorrectly) ran "greedy" with $h(n) = g(n)$, we would be expanding the smallest $g(n)$ first, i.e. we'd be equivalent to UCS.

11 Implementation Notes

11.1 Data Structure required

A typical search node:

```

1 Node:
2   state
3   parent
4   action
5   depth
6   g    // path cost so far

```

This matches our prior definition, and the EXPAND pseudocode storing parent, action, depth, and path cost.

Algorithm	Data Structure
BFS	Queue (FIFO)
DFS	Stack (LIFO)
UCS	Priority Queue keyed by $g(n)$
Greedy	Priority Queue keyed by $h(n)$
IDS	Repeated DLS runs; DLS itself uses recursion/stack up to depth l

Table 1: Fringe Implementations by Algorithm.

11.2 Repeated-State Checking

Whenever the state space graph has cycles (Romania), we need repeated-state checking to avoid infinite revisits.

12 Summary

- Formulation matters (8-queens example: enormous vs tiny state space).
- Search proceeds by managing **fringe** and **expanded** sets/list.
- Choice of "best" fringe node defines the algorithm:
 - BFS: shallowest depth
 - DFS: deepest depth
 - UCS: smallest $g(n)$

- Greedy: smallest $h(n)$
- IDS: DFS with increasing depth limits
- Trade-offs:
 - BFS: complete; optimal only under unit/constant step costs; huge memory.
 - DFS: low memory; not optimal; not complete in infinite depth.
 - UCS: complete (positive costs) + optimal; can be expensive if many small-cost steps exist.
 - IDS: "best default" when depth unknown: BFS-like guarantees with DFS-like memory.
 - Greedy: can be fast with good heuristics; not optimal; can loop without checks.